

A Multi-Simulation Bridge for IoT Digital Twins

Marco Picone
University of Modena and Reggio
Emilia
Italy

Samuele Burattini
University of Bologna
Italy

Marco Melloni
University of Modena and Reggio
Emilia
Italy

Prasad Talasila
Aarhus University
Denmark

Davide Ziglioli
University of Modena and Reggio
Emilia
Italy

Matteo Martinelli
University of Modena and Reggio
Emilia
Italy

Nicola Bicocchi
University of Modena and Reggio
Emilia
Italy

Alessandro Ricci
University of Bologna
Italy

Peter Gorm Larsen
Aarhus University
Denmark

Abstract

The increasing capabilities of Digital Twins (DTs) in the context of the Internet of Things (IoT) and Industrial IoT (IIoT) call for seamless integration with simulation platforms to support system design, validation, and real-time operation. This paper introduces the concept, design, and experimental evaluation of the *DT Simulation Bridge*—a software framework that enables diverse interaction patterns between active DTs and simulation environments. The framework supports both the DT development lifecycle and the incorporation of simulations during active operation. Through bidirectional data exchange, simulations can update DT models dynamically, while DTs provide real-time feedback to adapt simulation parameters. We describe the architectural design and core software components that ensure flexible interoperability and scalable deployment. Experimental results show that the DT Simulation Bridge enhances design agility, facilitates virtual commissioning, and supports live behavioral analysis under realistic conditions, demonstrating its effectiveness across a range of industrial scenarios.

CCS Concepts

• **Computing methodologies** → **Distributed simulation**; • **Networks** → **Internet of things**.

Keywords

Digital Twins, Industry, Simulation Bridge, Simulators, Agents

ACM Reference Format:

Marco Picone, Samuele Burattini, Marco Melloni, Prasad Talasila, Davide Ziglioli, Matteo Martinelli, Nicola Bicocchi, Alessandro Ricci, and Peter Gorm Larsen. 2025. A Multi-Simulation Bridge for IoT Digital Twins. In

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

With the widespread adoption of Internet of Things (IoT) and Industrial IoT (IIoT) technologies, Digital Twins (DTs) have evolved into *active* components—i.e., software entities that maintain continuous bidirectional communication with their physical assets [10, 12, 15]. This paradigm shift allows DTs to provide dynamic digital representations updated through sensor data streams, supporting functionalities such as real-time monitoring, anomaly detection, and adaptive control [4].

In this context, the integration of simulations within their architecture becomes increasingly critical. Simulations would enable DTs to perform “what-if” analyses, optimize performance parameters, and validate control strategies before applying them to the physical asset [3]. The combination of real-time data and virtual experimentation allows DTs to anticipate issues and adapt to changing conditions in complex environments. More in detail, we identified four primary motivations for integrating simulations into DTs:

(1) *Enabling predictive and prescriptive capabilities.* While IoT-based DTs are effective at capturing the current state of physical systems through real-time data, they are typically limited in projecting future behavior or reasoning under uncertainty. Simulation enhances DTs with predictive capabilities, supporting analysis under varying conditions such as equipment degradation, workload fluctuations, or environmental stressors [11]. This shifts DTs from descriptive digital replicas to prescriptive and prognostic systems capable of anticipating outcomes and triggering proactive interventions.

(2) *Coordinating heterogeneous simulations for complex systems.* Modeling complex systems often requires the orchestration of multiple, domain-specific simulations, each representing a different aspect of the physical system—e.g., mechanical dynamics, thermal effects, electrical behavior, or environmental interactions [7, 13]. These simulations may operate at different time scales, resolutions, or levels of abstraction and must be integrated to ensure coherence and consistency. This is outside the scope of this paper.

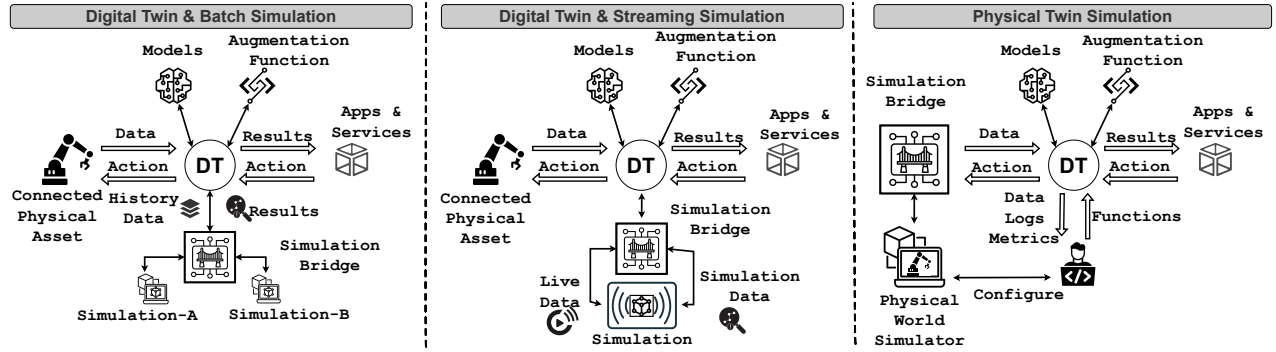


Figure 1: Overview of the Simulation Bridge enabling diverse interaction patterns between DTs and simulation environments.

(3) *Supporting DT development in the absence of the Physical Twin (PT).* In many scenarios – such as early-stage design, safety-critical applications, or high-cost manufacturing – the physical system may not yet exist, be inaccessible, or pose operational risks. In these cases, simulations – e.g., of the IoT system [6] – can act as surrogate PTs, generating mock behaviors and interaction patterns that allow the DT to be developed, tested, and validated in isolation [1].

(4) *Sharing simulation data across multiple DTs.* A single simulation model can serve multiple DTs, each potentially operating at different levels of abstraction and granularity. For instance, a simulation of a production line may provide relevant data to DTs representing individual subsystems, such as robots or sensors, each requiring different views, resolutions, and update semantics.

Despite these promising capabilities, the state of the art reflects only a partial integration of DTs and simulations [16]. This paper presents the *DT Simulation Bridge (DT-SB)*, a software framework designed to integrate DTs with multiple, heterogeneous simulation environments. The DT-SB enables bidirectional data exchange and supports diverse interaction patterns, allowing simulations to dynamically update the DT’s internal state, while DTs can influence and tune simulation parameters in real time. Unlike traditional co-simulation frameworks [9], the DT-SB connects a DT with multiple, independent simulations and acts solely as an intermediary. The coordination and aggregation of simulation results are delegated entirely to the DT, preserving architectural decoupling and maximizing flexibility. We describe the modular architecture and core software components of the DT-SB, emphasizing its interoperability with diverse simulation models and protocols. An experimental evaluation demonstrates the bridge’s effectiveness in accelerating iterative design cycles, facilitating virtual commissioning, and enabling live behavioral analysis under realistic operational conditions.

2 Related Work

Simulation is a key technology in DTs, envisioned as a tool that can be adopted across all phases of the PT lifecycle [3]. Despite the strong overlap between simulation and DTs making it difficult to draw a line between the technologies [16], it is becoming more and more clear that, especially in the IoT domain, DTs extend the capability of “static” simulations by incorporating real-time data [14].

Most DT simulations, though, integrate simulators and IoT data processing with custom architectures and ad-hoc methods. This lack of principled integration between the two worlds makes it hard to have a seamless experience especially when considering the possibility of using multiple simulation models within the same DT as we do in this paper.

The DT Simulation Bridge integrates and orchestrates multiple simulations with different models in a coherent framework. At a surface level, it is then similar to distributed simulation [8] and co-simulation [9]. There are, though, fundamental differences in the approach. Distributed simulation usually concerns the execution of a single complex simulation on distributed nodes. Differently, co-simulation explores the possibility of composing different simulation models of individual parts to simulate a complex system and coordinate the progress of time between the different independent simulation units. The purpose of the DT Simulation Bridge is, instead, to connect one DT with multiple simulations. The coordination and combination of such simulation results are expected to be managed by the DT and not by the bridge itself, which only acts as an intermediary. Nevertheless, despite these differences in purposes, there are some significant aspects from these two related areas which influence the design of the bridge.

Integrating simulations requires standardising the interaction across components. For distributed simulation, the High-Level Architecture (HLA) defines a set of constraints to design interoperable federations of simulations which can be executed on a distributed runtime infrastructure [5]. Despite in our case simulations do not need to exchange data between each other, the need for a standardized interface is essential to allow the DT to seamlessly interact with multiple different simulations. Similarly, in the co-simulation domain, the Functional Mockup Interface (FMI) is an industrial de facto standard [2] to bridge different simulation models. There, models of different nature may declare connection points to create input/output chains which combine the output of individual solvers. We follow this principle in the design of the DT-SB to support simulation models which can be updated over time with the data that is captured by the DT itself.

3 Simulation Bridge Interaction Patterns

The integration of DTs with simulation environments enables a range of interaction patterns designed to support different phases of the system lifecycle and corresponding operational requirements.

These interaction patterns are characterized by bidirectional data exchange and control flow, allowing DTs to incorporate predictive capabilities while simulations remain continuously informed by real-time contextual updates. Building on the DT-SB architecture, we identify three primary interaction paradigms: *Digital Twin & Batch Simulation*, *Digital Twin & Streaming Simulation*, and *Physical Twin Simulation*, as illustrated in Figure 1. Each paradigm serves distinct objectives, encompassing design validation, what-if scenario analysis, live system adaptation, and virtual commissioning. Together, they provide a flexible framework for coupling DTs with simulation processes across the development and operational spectrum.

Batch Simulation. Leverages one-shot simulation runs to analyze scenarios or validate design decisions related to the DT and its physical counterpart. Acting as an intermediary, the DT-SB receives inputs from the DT, such as the current system state, historical data, or scenario configuration parameters and subsequently triggers one or more simulation runs accordingly. These batch simulations execute to completion producing results that the DT-SB forwards back to the DT. The resulting simulation outcomes enable the DT to refine its models, develop predictive maintenance schedules, and recommend adjustments. This pattern is particularly well suited for use cases requiring comprehensive, offline evaluations, such as design validation and strategic scenario planning.

Streaming Simulation. Facilitates dynamic interactions between the DT and the simulation environment through continuous data exchange. In this mode, the DT streams high-frequency operational data to the DT-SB, which relays it to a connected simulation that dynamically adapts its model based on the incoming input. The simulation continuously produces output data that is streamed back to the DT through the DT-SB. This interaction enables live behavioral analysis and adaptive control. Furthermore, deviations between live DT data and simulation outputs can serve as early indicators of anomalies or faults, thereby enabling proactive intervention. This pattern is particularly well suited for applications demanding immediate feedback and continuous system optimization.

Physical Twin Simulation. Involves the use of a simulation environment that functions as a digital replica of the PT. This interaction mode is particularly valuable for virtual commissioning, operator training, and control logic testing, where system behaviors can be evaluated in a safe, simulated setting prior to deployment in the real world. In this pattern, the DT is unaware of the fact that it is communicating with a simulator and exchange data and control commands that are actuated in the simulated environment. DT developers can configure simulation parameters through the DT-SB in order to test the behavior of the DT under varying conditions. This pattern enables thorough testing and validation within a controlled virtual environment, significantly reducing the costs and risks associated with physical prototyping and on-site experimentation.

4 Simulation Bridge Architecture

The DT-SB coordinates heterogeneous and distributed simulations, providing a unified interface for DTs to access diverse simulation resources regardless of modeling and technological differences. The bridge serves as a middleware which routes simulation execution

requests, aggregates results, and returns them to clients. This enables seamless integration, reusability, and interoperability across simulation platforms.

Figure 2a illustrates the modular architecture of the DT-SB. At the heart of this architecture is the DT-SB, each simulator is integrated through a dedicated simulation agent (SA) module. SAs implement a standardized uniform API layer based on a simple application protocol. These agents are deployed within their respective simulation environments and are responsible for interpreting incoming messages, controlling the execution of simulation tasks, and aggregating the results for transmission back to the DT-SB. Each module of the DT-SB is designed to fulfill a specific function within the operational workflow:

Bridge Core: Serves as the central control unit of the DT-SB. It coordinates interactions among internal modules, monitors communication flows, and orchestrates message routing between external clients (e.g., DTs, PTs, simulators) and the distributed simulation environment.

Protocol Adapters (PAs): facilitate protocol translation and connection management between the DT-SB and external entities. They operate on both the north-bound interface (connecting to DTs and PTs) and the south-bound interface (connecting to SAs). PAs are implemented using a plugin architecture, allowing the system to be extended in a modular fashion.

Event Manager: DT-SB adopts an event-driven architecture, wherein all relevant actions within a PA (e.g., message transmission, reception, connection management) are internally represented as events. When a request or response is received by a PA, it is transformed into an internal event and processed by the Bridge Core.

Data Transformation Manager: is responsible for converting data formats for both incoming and outgoing communication. Transformation parameters are dynamically configurable to optimize message structures for transmission efficiency. The module supports configurable message granularity, enabling the system to manage both atomic events and composite result packets based on application requirements.

Configuration Manager: handles the validation and loading of system parameters, including supported protocols, access credentials, and routing rules. A centralized configuration approach facilitates maintainability and control. The configuration also specifies a whitelist of authorized DTs and SAs.

Logger: records internal operations, exceptions, and diagnostic messages. It also provides performance metrics and operational insights essential for monitoring and debugging the DT-SB.

Security Manager: oversees the generation and loading of TLS certificates, ensuring secure communication channels. It is also responsible for detecting and mitigating malicious activity, such as message flooding, that could lead to denial-of-service conditions targeting simulators.

4.1 Simulator Protocol

As DTs, DT-SB, and SAs operate in a distributed system, a dedicated *Simulation Protocol* (SP) serves as the unifying layer for managing simulation requests and responses. The protocol defines standard headers for request routing and result interpretation. PAs integrated into DT-SB provide their own delivery semantics; however,

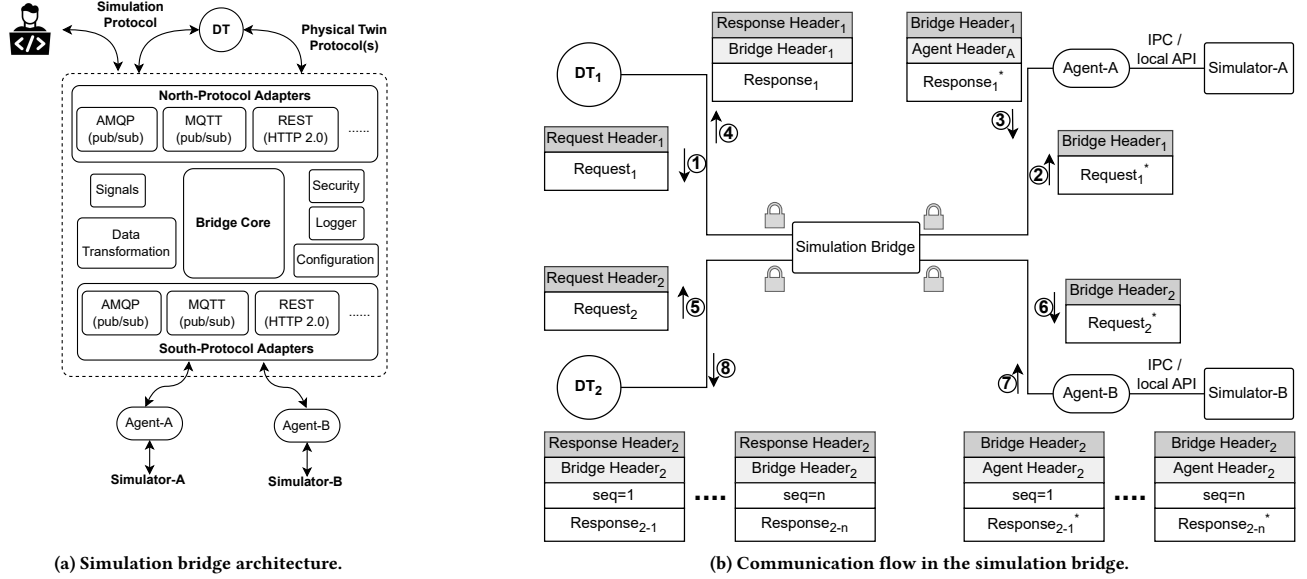


Figure 2: Architecture of the Simulation Bridge: (a) modular components supporting DT interaction and developer-driven PT simulation, and (b) communication flow between DTs, the bridge, and simulators for simulation control and integration.

best-effort delivery is universally supported, making it a practical foundation for interactions with SAs. Simulators can advertise the *Simulation Type (ST)* they support (an abstraction that defines expected behaviors and model-method combinations). DTs may then select one of the available STs, while the DT-SB is responsible for routing and translating the corresponding simulation requests to the appropriate SAs that provide the selected ST.

The DT-SB performs two key operations to mediate this interaction: (i) *data transformation* of simulation requests and responses to ensure syntactic and semantic consistency, and (ii) *protocol header replacement* to enforce proper message routing between DTs and the selected SAs. Figure 2b depicts the message exchange flow for both batch and streaming simulation modes. Secure variants of PAs are also illustrated. Numbered arrows indicate the sequence of message exchanges.

Batch Request. In this scenario, DT_1 issues a simulation request targeting a Simulation Type supported by *Simulator-A* (message m_1 , denoted as ① in Figure 2b). The request includes the model name, associated model parameters, simulator input values, and the specification of required outputs. Upon receiving the request, the DT-SB replaces the original request header with a bridge-specific header. This header replacement ensures the necessary abstraction and transparency between DTs and SAs, decoupling clients from simulator-specific implementations. The DT-SB also performs a data transformation step to normalize the request format before forwarding it to *Agent-A* (message m_2). *Agent-A* then transmits the transformed request to *Simulator-A*, which executes the simulation. The resulting outputs are sent back to the DT-SB via *Agent-A* (message m_3). *Agent-A* attaches a custom response header containing metadata relevant to the simulation run. Upon receiving

the response, the DT-SB applies a reverse data transformation tailored to the expectations of DT_1 , replaces the agent-specific header with a standardized response header derived from the original request metadata, and delivers the final message to DT_1 (message m_4). All headers in the message exchange carry metadata useful for distributed tracing, debugging, and performance analysis, thereby supporting observability and diagnostics across the system.

Streaming Request. In this scenario, DT_2 initiates a simulation request targeting *Simulator-B* (message m_5 , denoted as ⑤ in Figure 2b). As with batch requests, the DT-SB intercepts the request, applies the necessary data transformation, and replaces the header with a bridge-specific version before forwarding the request to *Agent-B* (message m_6). *Agent-B* then relays the request to *Simulator-B*, which begins executing the streaming simulation. Unlike batch simulations, streaming simulations generate a sequence of response messages over time. To ensure that responses are interpreted in the correct order, *Agent-B* attaches a sequence number to each outgoing message. This sequence number enables both DT-SB and DT_2 to preserve the temporal order of the response stream. *Agent-B* forwards the ordered sequence of responses to the DT-SB (denoted as multiple instances of message m_7), which in turn applies reverse data transformation and forwards the processed responses to DT_2 (messages m_8). In Figure 2b, these streaming responses have *seq* number field which is used by DT_2 to arrange the streaming responses in the correct order.

PT Simulation. For the simulation of PTs, the DT-SB is used by two distinct entities. On one side, the developer interacts with the bridge through the simulation protocol and the north-protocol adapters to configure and launch simulations, define the mapping of simulated entities, and specify how these entities are exposed to

the digital world via specific protocols, for example by mapping a simulation entity to MQTT. On the other side, the DT receives data from the simulated PTs through the configured protocols as if it were coming from the actual physical objects that will feed the DT in real deployments. Communication between the DT and the DT-SB is bidirectional: the DT not only receives telemetry data from the simulated PTs but can also send actions. These actions are translated by the bridge into commands for the simulation, modifying the behavior of the simulated PTs via interactions managed through the protocols and simulation agents enabled by the south-protocol adapters.

4.2 Routing Messages

A critical responsibility of the DT-SB is to map incoming simulation responses to their corresponding requests. This mapping is maintained using a dynamic routing table, with representative entries illustrated in Table 1.

Table 1: DT-SB's routing table.

PA_N	PA_S	DT	Sim. Type	Request-id	Timeout
MQTT	AMQP	DT_1	Octave	<i>hash#1</i>	60s
AMQP	MQTT	DT_1	Matlab	<i>hash#2</i>	60s
REST	AMQP	DT_2	Simul8	<i>hash#3</i>	120s

The column PA_N denotes the PA selected by the DT, while PA_S represents the PA associated with the Simulator Agent. In scenarios where the DT-SB supports only a single protocol on either the north-bound (PA_N) or south-bound (PA_S) interface, the corresponding column can be omitted to simplify the routing logic. The *Simulator Type* column identifies the class of simulations handled by a given SA. Each simulation request is uniquely identified by a RequestID, which may be generated using hash-based algorithms to ensure uniqueness and unpredictability. A timeout field is also included to define the expiration threshold for requests; expired entries are purged to maintain routing table efficiency. When a simulation response arrives, the tuple $\langle PA_S, ST, RequestID \rangle$ is extracted and matched against existing routing table entries. The identified row provides the target DT and the associated PA_N , allowing the DT-SB to correctly route the response back to the originating DT. This mechanism ensures correct correlation of simulation data even in asynchronous or distributed execution contexts.

5 Simulation Agents

5.1 Matlab Agent

The MATLAB Agent (SA_{MATLAB}) represents a decoupled simulation agent that integrates MATLAB's computational capabilities within the DT-SB architecture. The agent communicates with the DT-SB using the AMQP protocol, ensuring reliable and platform-agnostic messaging. Four distinct integration strategies were investigated, each offering different trade-offs in terms of latency, flexibility, and architectural coupling.

1. File-Based Exchange. In this approach, the simulator receives simulation inputs via files, executes the model, and stores the results in an output file. The SA_{MATLAB} reads the output file, encapsulates its contents as the response payload, and forwards the message to the DT-SB. This method offers high architectural decoupling but incurs substantial input/output overhead, making it unsuitable for streaming simulations due to poor performance and high latency.

2. TCP/IP Communication. This approach employs network-based IPC, enabling real-time interaction between the simulator and the SA_{MATLAB} . The simulator sends result data directly to the SA_{MATLAB} over a TCP/IP socket, formatted as JSON. Each result corresponds to a single message in the stream, making this method highly compatible with streaming simulation modes. The use of network IPC ensures that the MA and simulator remain decoupled at the operating system process level, supporting distributed deployment.

3. Shared Memory IPC. In this configuration, the SA_{MATLAB} and MATLAB exchange data via shared memory segments. Simulation requests and responses are written and read directly from volatile memory, resulting in extremely low latency and high throughput. However, this approach requires explicit synchronization using mutexes or locks to avoid race conditions and deadlocks. Additionally, shared memory access necessitates strict control over memory layout, reducing flexibility.

4. MATLAB Engine API. This method utilizes the MATLAB Engine API to directly invoke MATLAB functions from Python. While it simplifies invocation of MATLAB computations, it imposes structural constraints on model definitions and requires the simulator to conform to specific function-call semantics, thus reducing modularity and reusability.

A comprehensive evaluation of these four approaches has been conducted to determine the most suitable integration strategy for *batch* and *streaming* simulation modes. Although shared memory IPC achieves the lowest latency, it required rigid memory management and offers limited flexibility. Ultimately, the Matlab Engine API approach has been selected for batch simulation. The TCP/IP approach has been selected as the preferred integration method for streaming mode simulation due to its balance of modularity, ease of integration, and suitability for distributed architectures.

5.2 AnyLogic Agent

AnyLogic¹, is a flexible simulation platform that combines agent-based, discrete-event, and system dynamics modeling. This versatility makes it ideal for creating detailed digital representations of industrial CPSs. We hence develop the agent to support the PT simulation pattern.

The AnyLogic Agent has two components: 1) an interface to the DT-SB which accepts simulation requests and configures the AnyLogic scenario accordingly. Differently from the other patterns, these requests come from the DT developer who wants to test the DT behaviour on a simulated PT; 2) a general purpose *entity*² that can be embedded within the simulation to receive messages from

¹AnyLogic: <https://www.anylogic.com/>

²We use *entity* here to avoid overloading the term agent. Since AnyLogic is an agent-based simulator, all entities within the simulations are modeled as agents which can exchange messages using built-in mechanisms.

other entities and serialize them on a network socket to output a data stream of relevant events happening within the simulation. The simulation data stream is then received from the south protocol adapters, and routed to the physical interface of the DT with the north protocol adapters (Fig. 2a).

The AnyLogic data stream carries information about which simulation entity generated it, which can be used by the bridge to map the corresponding update on a different protocol for the DT. This design is needed to emulate scenarios where data about one or more PTs is sent over different communication protocols (e.g. a sensor network), making it possible to test the DT simply by changing its configuration to point to the emulated communication channels and leaving all the connection handling and message processing logic unaltered. The connection is bi-directional, to allow the DT to send control commands and apply them onto the simulated PT.

Table 2: Performance of DT-SB for different protocols.

Protocol	Overhead	STD	5% Threshold	95% Threshold
MQTT	2.47ms	10.49ms	1.69ms	11.98ms
AMQP	1.95ms	0.99ms	1.12ms	3.69ms
REST	4.63ms	4.72ms	2.92ms	11.42ms

6 Experimental Evaluation

To evaluate the performance of the proposed DT-SB and its integration with simulation agents, we conducted a series of experiments focusing on protocol overhead and agent-level performance. All tests were executed on an Apple MacBook Air (2023) equipped with an Apple Silicon M2 processor and 8 GB of unified memory. MATLAB was natively installed on macOS, and Python profiling modules were used to gather performance metrics.

6.1 Simulation Bridge Overhead

DT-SB Overhead. The DT-SB was implemented to support multi-protocol communication on its north-bound interface (facing DTs), including MQTT, AMQP, and HTTP/2.0. Its south-bound interface (facing simulation agents) currently supports AMQP. We evaluated the processing time overhead introduced by the DT-SB core, explicitly excluding the contribution of individual protocol adapters. Table 2 shows the descriptive statistics of measured processing time overhead for MQTT, AMQP, and HTTP/2.0 protocol adapters. These results confirm that DT-SB can reliably handle heterogeneous PAs with minimal latency in request forwarding and agent selection.

Matlab Agent Overhead. To assess the performance of the MATLAB agent independently of simulation model complexity, we used lightweight test cases to isolate the agent’s intrinsic processing overhead. Each simulation request was assigned a unique *Request ID*, enabling precise tracking of request-response cycles. For streaming mode experiments, different time steps were used to vary the frequency of data exchange, simulating increased communication loads. The overhead refers to the time spent by the agent to process a simulation request, excluding MATLAB startup time

and simulation runtime. This metric captures the cost of request parsing, file operations, and inter-process communication. In batch mode, the agent exhibited a mean overhead of 2.7 ms. In streaming mode, the mean overhead was 3.7 ms. Table 3 reports the measured overhead for different streaming frequencies, showing that agent performance remains stable across varying time steps. These findings confirm that *SA_{MATLAB}* introduces minimal latency, making it suitable for both batch and real-time streaming applications.

Table 3: Matlab Agent’s overhead for different frequencies.

Streaming Frequency (ms)	10	50	70	100	150	200
Agent Overhead (ms)	2.2	4.1	5.6	4.9	3.2	3.0

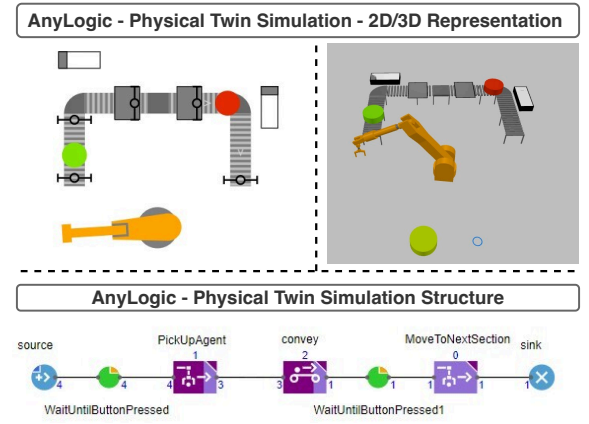


Figure 3: AnyLogic PT simulation.

6.2 Physical Twin Simulation

The simulated environment was developed using the DT-SB integrated agent for AnyLogic to support DT creation for a microfactory scenario, featuring an indexed line with a conveyor, workstations, and a robot handling parts between loading areas, as shown in Figure 3.

The two-dimensional model, includes conveyors reporting status, photocells detecting product passage, and transfer stations with custom switches emulating pushers. Workstations have virtual actuators reporting status, and the central robot, modeled via the *Robot* element, manages item positioning and state updates. The simulation integrates all key entities and focuses on assessing the communication overhead of bidirectional data exchange with the DT via the DT-SB.

Process logic uses a block diagram: agents are generated by a *Source* block, controlled by a *Hold* block linked to UI buttons, processed by a *ProcessByRobot* block, moved via a *Convey* block, and removed by a *Sink* block. The DT processes data as if from real PTs. The measured communication overhead from AnyLogic to the DT via MQTT averaged 6.5 ms with a standard deviation of 10 ms.

This setup offers a controlled, repeatable environment for testing DTs, supporting experimentation with PT conditions, communication protocols, synchronization, and CPS integration strategies.

7 Conclusion

In this paper, we tackled the integration of heterogeneous simulations with DTs via a general-purpose middleware framework. We analyzed key interaction patterns—*batch*, *streaming*, and *PT simulation*—supporting various DT lifecycle use cases. We proposed the *DT Simulation Bridge (DT-SB)* architecture and a lightweight protocol that ensures a uniform interface between DTs and diverse simulation environments. The DT-SB enables data translation, protocol adaptation, and interoperability across platforms. We validated our solution through two case studies: (i) MATLAB-based streaming performance evaluation; and (ii) AnyLogic-based PT simulation of a cyber-physical scenario. Future work will focus on classifying integration patterns, extending to co-simulation and hybrid setups, refining synchronization, and exploring adaptive orchestration in distributed environments.

References

- [1] Abderrahim Ait-Alla, Markus Kreutz, Daniel Rippel, Michael Lütjen, and Michael Freitag. 2019. Simulation-based Analysis of the Interaction of a Physical and a Digital Twin in a Cyber-Physical Production System. *IFAC-PapersOnLine* 52, 13 (2019), 1331–1336. doi:10.1016/j.ifacol.2019.11.383 9th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2019.
- [2] Torsten Blochwitz. 2014. Functional Mock-up Interface for Model Exchange and Co-Simulation (version 2.0). <https://www.fmi-standard.org/downloads>.
- [3] Stefan Boschert and Roland Rosen. 2016. *Digital Twin—The Simulation Aspect*. Springer International Publishing, Cham, 59–74. doi:10.1007/978-3-319-32156-1_5
- [4] Chiara Cimino, Elisa Negri, and Luca Fumagalli. 2019. Review of digital twin applications in manufacturing. *Computers in Industry* 113 (2019), 103130.
- [5] J.S. Dahmann. 1997. High Level Architecture for simulation. In *Proceedings First International Workshop on Distributed Interactive Simulation and Real Time Applications*. 9–14. doi:10.1109/IDSRTA.1997.568652
- [6] Gabriele D'Angelo, Stefano Ferretti, and Vittorio Ghini. 2016. Simulation of the Internet of Things. In *2016 International Conference on High Performance Computing & Simulation (HPCS)*. 1–8. doi:10.1109/HPCSIm.2016.7568309
- [7] John Fitzgerald, Peter Gorm Larsen, and Ken Pierce. 2019. *Multi-modelling and Co-simulation in the Engineering of Cyber-Physical Systems: Towards the Digital Twin*. Springer International Publishing, Cham, 40–55. doi:10.1007/978-3-030-30985-5_4
- [8] Richard Fujimoto. 2000. *Parallel and distributed simulation systems*. Wiley.
- [9] Cláudio Gomes, Casper Thule, David Broman, Peter Gorm Larsen, and Hans Vangheluwe. 2018. Co-simulation: a Survey. *ACM Comput. Surv.* 51, 3 (May 2018), 49:1–49:33.
- [10] Michael Grieves. 2014. Digital twin: manufacturing excellence through virtual factory replication. *White paper* 1, 2014 (2014), 1–7.
- [11] Jan-Felix Klein and Kai Furmans. 2026. A study on the predictive capabilities of digital twins for object transfers in a remanufacturing demonstration environment. *Robotics and Computer-Integrated Manufacturing* 97 (2026), 103063. doi:10.1016/j.rcim.2025.103063
- [12] Werner Kritzinger, Matthias Karner, Georg Traar, Jan Henjes, and Wilfried Sihh. 2018. Digital Twin in manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine* 51, 11 (2018), 1016 – 1022. doi:10.1016/j.ifacol.2018.08.474 16th IFAC Symp. on Information Control Problems in Manufacturing 2018.
- [13] Xiaonan Lai, Shuo Wang, Zhenggang Guo, Chao Zhang, Wei Sun, and Xueguan Song. 2021. Designing a Shape-Performance Integrated Digital Twin Based on Multiple Models and Dynamic Data: A Boom Crane Example. *Journal of Mechanical Design* 143, 7 (02 2021), 071703. doi:10.1115/1.4049861
- [14] Rakesh Kumar Phanden, Priavrat Sharma, and Anubhav Dubey. 2021. A review on simulation in digital twin for aerospace, manufacturing and robotics. *Materials Today: Proceedings* 38 (2021), 174–178. doi:10.1016/j.matpr.2020.06.446 2nd International Conference on Future Learning Aspects of Mechanical Engineering.
- [15] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee. 2019. Digital Twin in Industry: State-of-the-Art. *IEEE Transactions on Industrial Informatics* 15, 4 (2019), 2405–2415. doi:10.1109/TII.2018.2873186
- [16] Ana Wooley, Daniel F. Silva, and Julia Bitencourt. 2023. When is a simulation a digital twin? A systematic literature review. *Manufacturing Letters* 35 (2023), 940–951. doi:10.1016/j.mfglet.2023.08.014 51st SME North American Manufacturing Research Conference (NAMRC 51).